

3교시: Prometheus Target 확인

W4D3 Lesson 3 Prometheus Targets

Prometheus는 ServiceMonitor / PodMonitor를 통해 대상(Target)을 발견하고 /metrics 엔드포인트를 스크랩합니다.

1. Prometheus 타겟 발견 흐름

2. ServiceMonitor / PodMonitor 예시

ServiceMonitor 예시 (api 서버 예드포인트)

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: api-sm
  namespace: monitoring
spec:
  selector:
    matchLabels:
      http_requests_total
      process_cpu_seconds_total
      container_memory_usage_bytes
      kube_pod_info
  endpoints:
    - port: http-metrics
      path: /metrics
```

서비스의 Label(app=api)과 일치하는 Service를 찾아 예드포인트 수집

PodMonitor 예시 (노드 익스포터)

```
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: node-exporter-pm
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-exporter
  namespaceSelector:
    matchNames: [ app-ns ]
  podMetricsEndpoints:
    - port: metrics
      path: /metrics
```

Pod의 Label(app=node-exporter)과 일치하는 Pod를 직접 모니터링

엔드포인트(Endpoints) 구조

Service가 선택한 Pod의 IP-Port 목록 제공

```
api-svc Endpoints 예시
10.244.1.12:8080
10.244.1.13:8080
10.244.1.14:8080
...
```

Pod IP-Port → /metrics 스크랩 대상

3. Prometheus Targets 상태 예시

상태	Job	Namespace	Service/Pod	Endpoint	Labels (주요)	Last Scrape	Scrape Duration	State Reason
UP	api-sm	app-ns	api-svc	10.244.1.12:8080/metrics	job="api", namespace="app-ns", service="api-svc", pod="api-6f8b9d7c5-abc12"	5s ago	28ms	-
UP	frontend-sm	app-ns	frontend-svc	10.244.2.10:8080/metrics	job="frontend", namespace="app-ns", service="frontend-svc", pod="frontend-bd8f9d6-ef12"	4s ago	16ms	-
DOWN	db-sm	db-ns	postgres-svc	-	job="postgres", namespace="db-ns", service="postgres-svc"	-	-	-
DOWN	api-sm	app-ns	api-svc	10.244.1.15:8080/metrics	job="api", namespace="app-ns", service="api-svc", pod="api-6f8b9d7c5-def34"	10s ago	-	content deadline exceeded
DOWN	node-exporter-gm	monitoring	node-exporter	10.244.0.23:9100/metrics	job="node-exporter", namespace="monitoring", pod="node-exporter-abcde"	12s ago	-	404 Not Found

Targets UI: <http://x-prometheus-9090/targets> → 각 Job의 대상 상태와 오류 원인을 확인할 수 있습니다.

4. 타겟 DOWN 주요 원인과 해결 방법

원인 (State Reason)	설명 / 해결 방법
no healthy targets	• Service의 Endpoints가 비어 있음 • Pod가 Ready가 아니거나 라벨이 일치하지 않음 → Endpoints 확인, Pod 라벨 Ready 상태 점검
content deadline exceeded	• 스크립 타임아웃 (네트워크 지연, 포트 응답 없음) • 네트워크 연결, Pod 로그, interval/timeout 조정
404 Not Found	• /metrics 경로가 존재하지 않음 • path 설정 확인, 애플리케이션 예드포인트 엔드포인트 확인
connection refused	• 포트가 열려있지 않음 • 컨테이너/서비스 포트, 이름(port) 설정 확인
network policy blocked	• NetworkPolicy로 Prometheus 접근 차단 • 모니터링 네임스페이스에서 접근 허용 규칙 추가

5. 빠른 진단 명령어

```
# Service/Endpoints 확인
kubectl get svc -n app-ns api-svc
kubectl get endpoints -n app-ns api-svc -o wide

# Pod 상태와 라벨 확인
kubectl get pods -n app-ns -l app=api --show-labels

# ServiceMonitor/PodMonitor 확인
kubectl get servicemonitor -A
kubectl describe servicemonitor api-sm -n monitoring
```

6. 스크립 설정 체크포인트

- 서비스/Pod 라벨과 selector 일치
- namespaceSelector 설정 확인
- endpoints.port 이름 또는 숫자 포트 정확성
- path: /metrics 경로 문제 여부
- Pod Ready 상태 (readinessProbe 통과)
- NetworkPolicy에서 Prometheus 허용
- interval / timeout 설정 적절성

7. 흐름 요약

8. 모범 사례 (Best Practices)

- 명확한 라벨 설정 (app, component 등)
- 서비스는 안정적인 port 이름 사용
- /metrics 엔드포인트 표준화
- 적절한 interval(15s~60s) 설정
- 모니터링 전용 네임스페이스로 분리
- 문제 발생 시 Targets UI를 먼저 확인

9. 증거(Evidence) 체크리스트

- Targets UI 캡처 (UP/DOWN 상태)
- Service/Endpoints 출력 첨부
- Pod 라벨/Ready 상태 확인 결과
- ServiceMonitor/PodMonitor YAML
- 오류 원인 및 해결 조치 기록

Targets가 UP인지만, DOWN인지 확인하고 원인별로 차근차근 해결해 보세요!

수업 목표

- Prometheus target의 UP/DOWN 의미를 설명한다.
- ServiceMonitor, PodMonitor가 target을 만드는 방식을 이해한다.
- target down 원인을 namespace, selector, port, endpoint 기준으로 나눈다.

Target이란 무엇인가

Prometheus는 endpoint를 주기적으로 scrape한다.

Prometheus

- > target endpoint scrape
- > metric 저장
- > PromQL query

target이 up이어야 metric이 들어온다.

UI에서 확인

Prometheus 접속:

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-prometheus 9090:9090
```

브라우저:

```
http://localhost:9090/targets
```

확인할 것:

항목	의미
State UP	scrape 성공
State DOWN	scrape 실패
Last Scrape	마지막 수집 시각
Error	실패 이유
Labels	job, namespace, service, pod

PromQL로 확인

```
up
```

namespace별 target:

```
up{namespace="monitoring"}
```

값 해석:

값	의미
1	target UP
0	target DOWN
series 없음	target 자체가 discovery되지 않음

kind/local 검증에서 볼 수 있는 예:

```
up{job="kube-scheduler"} 0
up{job="kube-apiserver"} 1
up{job="kube-state-metrics"} 1
```

kube-scheduler가 DOWN이라고 해서 Prometheus 설치가 실패했다는 뜻은 아니다. local kind에서는 control plane component endpoint 노출 방식 때문에 일부 target이 기대와 다르게 보일 수 있다. 이때는 up 값만 보고 결론 내리지 말고 target error, ServiceMonitor, Endpoints를 같이 본다.

up == 0과 series 없음의 차이

둘은 다르다.

```
up{job="kube-state-metrics"} == 0
```

은 target은 발견됐지만 scrape가 실패했다는 뜻이다.

반면 query 결과가 아예 없으면 Prometheus가 target을 발견하지 못했을 수 있다.

상태	의미	볼 것
up=1	scrape 성공	정상
up=0	target 발견, scrape 실패	target error
결과 없음	target discovery 실패	ServiceMonitor/selector

이 차이를 모르면 target down과 설정 누락을 같은 문제로 착각한다.

ServiceMonitor와 PodMonitor

Prometheus Operator는 ServiceMonitor/PodMonitor를 읽어 scrape target을 만든다.

```
kubectl get servicemonitor,podmonitor -A
```

예상 출력:

```
monitoring    kube-prometheus-stack-kubelet
monitoring    kube-prometheus-stack-prometheus
monitoring    kube-prometheus-stack-grafana
```

ServiceMonitor가 target을 만들려면 다음이 맞아야 한다.

조건	확인
namespace selector	대상 namespace를 보는가
service selector	label이 맞는가
endpoint port name	Service port 이름이 맞는가
endpoint path	/metrics가 맞는가
Pod readiness/network	scrape 가능한가

port 이름이 중요한 이유

ServiceMonitor는 보통 Service port 이름을 참조한다.

```
endpoints:
  - port: http-metrics
```

그런데 Service가 이렇게 되어 있으면 target이 안 잡힐 수 있다.

```
ports:
  - name: http
    port: 8080
```

이 경우 http-metrics라는 port 이름이 없기 때문이다. Prometheus target 문제에서 port name mismatch는 흔하다.

target error 예시

Error	해석
context deadline exceeded	timeout, network, policy
connection refused	endpoint는 있으나 port가 안 열림
server returned HTTP status 404	/metrics path 없음
no such host	DNS 문제
server returned HTTP status 403	auth/RBAC/proxy 문제

target down 원인

원인	증상	확인
Service 없음	target discovery 안 됨	kubectl get svc
selector 불일치	endpoint 없음	kubectl get endpoints
port 이름 불일치	target 생성 실패	ServiceMonitor endpoint port
/metrics 없음	scrape HTTP error	target error message
NetworkPolicy	timeout	policy, DNS, egress
Pod down	connection refused	Pod READY/logs

kind에서 control plane target이 DOWN일 때 확인 순서:

```
kubectl -n monitoring get servicemonitor | grep scheduler
kubectl -n kube-system get endpoints kube-scheduler
kubectl -n monitoring logs deploy/kube-prometheus-stack-operator
```

운영 판단:

상황	판단
Grafana/Prometheus/kube-state-metrics가 정상	stack 설치 자체는 성공
특정 target만 up=0	target endpoint 또는 scrape 설정 문제
대부분 target이 없음	selector/CRD/operator 문제

Target Down 분석 템플릿

```
## Target
- job:
- namespace:
- endpoint:
- state:
- error:

## Kubernetes object
- Service:
- Endpoint:
- Pod READY:
- ServiceMonitor selector/port:

## 판단
- 원인 후보:
- 다음 확인:
```

Evidence Note

```
# W4D3S3 Prometheus targets
- UP target 1개:
- DOWN target 또는 없었던 target:
- target label:
- target error:
- 관련 ServiceMonitor:
```

한 줄 요약

Prometheus target이 없거나 DOWN이면 dashboard보다 먼저 discovery, Service, Endpoint, port를 확인한다.